LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4&5 The Core of OOP & Encapsulation

Disusun Oleh:

Wisnu Bimantoro 105224045

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Sistem reservasi pada jaringan perhotelan saat ini sering kali menghadapi berbagai kendala operasional yang merugikan, baik dari sisi efisiensi maupun finansial. Beberapa masalah utama yang sering muncul adalah terjadinya bug operasional, seperti jumlah tamu yang melebihi kapasitas kamar (overcapacity) serta penyalahgunaan kode voucher diskon oleh tamu yang tidak memenuhi kriteria. Jika celah ini dibiarkan, maka integritas data dan kualitas layanan hotel akan menurun.

Laporan praktikum ini berfokus pada perancangan ulang arsitektur sistem reservasi melalui pembuatan blueprint kelas KamarHotel. Pendekatan utama yang digunakan adalah penerapan dua pilar penting dalam Object-Oriented Programming (OOP):

1. Enkapsulasi (Encapsulation): Digunakan untuk melindungi atribut krusial kamar agar tidak dapat diakses atau dimodifikasi secara langsung dari luar kelas. Dengan menerapkan modifier private dan menyediakan metode Setter yang disertai validasi ketat, sistem bertindak sebagai "pengawas" otomatis terhadap kesalahan input (human error).
2. Overloading Metode (Method Overloading): Diterapkan untuk memberikan fleksibilitas pada perilaku sistem, seperti menangani berbagai skenario pemesanan (dengan atau tanpa parameter jumlah tamu) serta perhitungan tagihan yang adaptif terhadap penggunaan kode promo.

Melalui pembaruan arsitektur ini, diharapkan sistem reservasi menjadi lebih tangguh, presisi, dan mampu meminimalisir kesalahan operasional di meja resepsionis.

.

II. Variabel

No	**Nama Variabel**	**Tipe Data**
1	nomorKamar	String
2	tipeKamar	String
3.	kapasitasMaksimal	Int
4.	hargaPerMalam	double
5.	isTersedia	Boolean

iii. **Constructor dan Method**

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel(String nomor, String tipe, int kapasitas)	constructor	Membuat objek kamar baru untuk pendaftaran kilat. Harga otomatis diatur ke 0 dan status tersedia diset true).
2	KamarHotel(String nomor, String tipe, int kapasitas, double harga)	constructor	Membuat objek kamar dengan data lengkap (termasuk harga per malam). Status tersedia langsung diset true.
3.	getNomorKamar(), getTipeKamar(), dll.	Getter (Functional)	Mengambil atau membaca nilai dari atribut yang dikunci (private).
4.	setTipeKamar(String)	procedural	Mengubah tipe kamar dengan validasi: hanya menerima "Reguler", "Premium", atau "Suite". Selain itu dipaksa jadi "Reguler”.
5.	setHargaPerMalam(double)	procedural	Mengubah tarif harga per malam dengan validasi bisnis batas bawah (menolak angka di bawah 50.000).
6.	pesanKamar()	procedural	Memesan kamar tanpa syarat tambahan. Status kamar berubah jadi false kalau masih kosong.
7.	pesanKamar(int jumlahTamu)	procedural	Memesan kamar dengan cek kapasitas. Kalau tamu kebanyakan (*overcapacity*), pesanan bakal ditolak otomatis.
8.	batalPesan()	Procedural	Membatalkan pesanan dan mengembalikan status kamar jadi tersedia lagi (true)
9.	hitungTotalBayar(int jumlahMalam)	Functional	Menghitung total biaya menginap dengan harga normal (Harga $\times$ Malam).
10.	hitungTotalBayar(int jumlahMalam, String voucher)	functional	Menghitung biaya dengan cek voucher "PROMO". Kalau syarat minimal 3 malam oke, dapet diskon 20%.
11.	cetakInfo()	Procedural	Menampilkan struk berisi semua detail informasi kamar ke layar secara rapi.

IV. Dokumentasi dan Pembahasan Code

Pembahasan Kode

```
1  public class KamarHotel {
2
3      private String nomorKamar;
4      private String tipeKamar;
5      private int kapasitasMaksimal;
6      private double hargaPerMalam;
7      private boolean isTersedia;
8
9      public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
10         this.nomorKamar = nomorKamar;
11         setTipeKamar(tipeKamar);
12         this.kapasitasMaksimal = kapasitasMaksimal;
13         this.hargaPerMalam = 0;
14         this.isTersedia = true;
15     }
```

1. Atribut dan Enkapsulasi (Baris 3-7)

Di bagian awal kelas, kita mendefinisikan "identitas" atau atribut dari kamar hotel:

- Baris 3-7: Kita menggunakan modifier private pada variabel nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam, dan isTersedia.
- Kenapa pakai private? Ini adalah penerapan Encapsulation. Tujuannya supaya variabel ini nggak bisa diubah sembarangan dari luar kelas. Jadi, data kamar tetap aman dan hanya bisa diakses lewat jalur resmi (Getter dan Setter).

2. Constructor Pendaftaran Kilat (Baris 9-15)

Bagian ini adalah Constructor pertama yang dirancang untuk proses registrasi kamar yang cepat.

- Parameter: Hanya menerima tiga input utama yaitu nomorKamar, tipeKamar, dan kapasitasMaksimal.
- Logika Otomatis:
    - Pada baris 11, kita memanggil setTipeKamar(tipeKamar). Ini cerdas karena kita nggak langsung simpan datanya, tapi dilempar dulu ke metode Setter buat dicek apakah tipenya valid atau nggak.
    - Sesuai permintaan modul, sistem otomatis mengatur hargaPerMalam menjadi 0.
    - Status isTersedia langsung diset true, artinya kamar baru ini siap untuk dipesan.

```
17     public KamarHotel(String nomorKamar, String tipeKamar,
18                       int kapasitasMaksimal, double hargaPerMalam) {
19
20         this.nomorKamar = nomorKamar;
21         setTipeKamar(tipeKamar);
22         this.kapasitasMaksimal = kapasitasMaksimal;
23         setHargaPerMalam(hargaPerMalam);
24         this.isTersedia = true;
25     }
```

```
27     public String getNomorKamar() {
28         return nomorKamar;
29     }
30
31     public String getTipeKamar() {
32         return tipeKamar;
33     }
34
35     public int getKapasitasMaksimal() {
36         return kapasitasMaksimal;
37     }
38
39     public double getHargaPerMalam() {
40         return hargaPerMalam;
41     }
42
43     public boolean getIsTersedia() {
44         return isTersedia;
45     }
```

.

3. Constructor Pendaftaran Lengkap (Baris 17-25)

Berbeda dengan yang pertama, constructor ini buat tamu yang mau datanya langsung beres semua.

- Baris 21 & 23: Perhatikan kalau di sini kita nggak pakai this.tipeKamar = ... secara langsung. Kita tetap manggil setTipeKamar dan setHargaPerMalam.
- Tujuannya? Supaya kalau ada admin yang ngawur masukin harga pas pendaftaran awal, sistem tetap bisa "nyaring" lewat metode validasi yang sudah kita buat. Tetap aman sejak detik pertama objek dibuat!

4. Metode Getter (Baris 27-45)

Karena tadi semua atributnya kita "kunci" pakai private, maka baris-baris ini adalah satu-satunya cara buat kita ngelihat isi datanya.

- Semua metode ini pakai jenis Functional karena mereka punya return value.
- Jadi kalau kelas lain mau nanya "Kamar ini tipenya apa?", mereka bakal manggil getTipeKamar() dan si sistem bakal ngasih tau isinya tanpa membiarkan orang itu ngerubah-rubah isinya.
-

```
47      public void setTipeKamar(String tipeKamar) {
48
49          if (tipeKamar.equalsIgnoreCase("Reguler") ||
50              tipeKamar.equalsIgnoreCase("Premium") ||
51              tipeKamar.equalsIgnoreCase("Suite")) {
52
53              this.tipeKamar = tipeKamar;
54          } else {
55              System.out.println("Tipe kamar tidak valid, Diubah menjadi Reguler.");
56              this.tipeKamar = "Reguler";
57          }
58      }
59
```

```
60      public void setHargaPerMalam(double hargaPerMalam) {
61
62          if (hargaPerMalam < 50000) {
63              System.out.println("Harga terlalu rendah, Diubah menjadi 50000.");
64              this.hargaPerMalam = 50000;
65          } else {
66              this.hargaPerMalam = hargaPerMalam;
67          }
68      }
```

5. Validasi Ketat pada Metode Setter (Baris 47-68)

Bagian ini menunjukkan bahwa enkapsulasi bukan cuma soal menyembunyikan data, tapi juga mengontrol bagaimana data itu boleh diubah.

- setTipeKamar (Baris 47-58):
  - Sistem menggunakan fungsi equalsIgnoreCase supaya input seperti "reguler", "REGULER", atau "Reguler" tetap dianggap sah.
  - Jika input yang masuk di luar pilihan "Reguler", "Premium", atau "Suite", maka blok else akan bekerja. Sistem bakal mencetak peringatan ke layar dan secara otomatis memaksa nilai tipe kamar tersebut menjadi "Reguler" agar operasional hotel tetap berjalan standar.
- setHargaPerMalam (Baris 60-68):
  - Di sini kita memasang batas minimum harga. Jika ada staf yang memasukkan angka di bawah 50.000 (termasuk angka negatif), sistem akan langsung mendeteksi kondisi tersebut pada baris 62.
  - Sistem bakal menolak angka tersebut, mengeluarkan pesan teguran, dan secara otomatis menetapkan harganya menjadi 50.000 sebagai jaring pengaman finansial hotel.

.

```
70      public void pesanKamar() {
71          if (isTersedia) {
72              isTersedia = false;
73              System.out.println("Kamar berhasil dipesan.");
74          } else {
75              System.out.println("Kamar sudah terisi!");
76          }
77      }
78
79      public void pesanKamar(int jumlahTamu) {
80
81          if (!isTersedia) {
82              System.out.println("Kamar sudah dipesan");
83              return;
84          }
85
86          if (jumlahTamu > kapasitasMaksimal) {
87              System.out.println("Jumlah tamu melebihi kapasitas, Pesanan ditolak.");
88          } else {
89              isTersedia = false;
90              System.out.println("Pemesanan berhasil untuk "
91                      + jumlahTamu + " tamu.");
92          }
93      }
```

6. Reservasi dengan Method Overloading (Baris 70-93)

Pada bagian ini, kita membuat sistem pemesanan yang fleksibel untuk menangani berbagai situasi di meja resepsionis.

- Pemesanan Standar (Baris 70-77):
    - Metode pesanKamar() tanpa parameter ini digunakan untuk proses cepat.
    - Logikanya sangat sederhana: sistem hanya mengecek apakah kamar tersebut tersedia (isTersedia).
    - Jika kosong, status kamar langsung diubah menjadi false (terisi) dan mencetak pesan sukses. Namun, jika sudah terisi, sistem akan memberikan peringatan bahwa kamar tersebut tidak bisa dipesan lagi.
- Pemesanan dengan Validasi Tamu (Baris 79-93):
    - Metode pesanKamar(int jumlahTamu) ini jauh lebih ketat karena menerima input jumlah orang yang akan menginap.
    - Pengecekan Pertama (Baris 81-84): Sistem memastikan dulu apakah kamarnya sedang kosong atau tidak. Jika sudah dipesan, proses langsung berhenti (return).
    - Pengecekan Kedua (Baris 86-92): Ini adalah fitur anti-overcapacity. Sistem akan membandingkan jumlahTamu dengan kapasitasMaksimal kamar.
    - Jika tamu yang datang terlalu banyak, pesanan otomatis ditolak dengan pesan teguran khusus agar operasional hotel tetap aman dan sesuai aturan muatan kamar.

.

```
95          public void batalPesan() {
96              isTersedia = true;
97              System.out.println("Pesanan dibatalkan, kamar tersedia kembali.");
98          }
99
100         public double hitungTotalBayar(int jumlahMalam) {
101             return jumlahMalam * hargaPerMalam;
102         }
103
104         public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
105
106             double total = jumlahMalam * hargaPerMalam;
107
108             if (kodeVoucher.equals("PROMO") && jumlahMalam >= 3) {
109                 System.out.println("Voucher berhasil, Diskon 20% diberikan.");
110                 return total * 0.8;
111             } else {
112                 System.out.println("Voucher gagal digunakan.");
113                 return total;
114             }
115         }
```

7. Pembatalan dan Logika Penagihan (Baris 95-115)

Bagian ini menutup fungsionalitas utama dari kelas kamar dengan menyediakan fitur pembatalan serta perhitungan biaya yang fleksibel.

- batalPesan (Baris 95-98):
    - Metode ini berfungsi untuk me-reset status kamar.
    - Saat dieksekusi, variabel isTersedia akan diubah kembali menjadi true. Ini memastikan bahwa setelah tamu membatalkan atau check-out, kamar tersebut bisa muncul kembali di sistem sebagai kamar yang kosong dan siap dipesan oleh tamu berikutnya.
- Perhitungan Tagihan Standar (Baris 100-102):
    - Ini adalah versi simpel dari hitungTotalBayar.
    - Sistem hanya mengambil parameter jumlahMalam lalu mengalikannya dengan hargaPerMalam untuk mendapatkan total tagihan normal.
- Perhitungan dengan Voucher (Baris 104-115):
    - Di sini kita menggunakan Method Overloading lagi untuk sistem penagihan.
    - Validasi Promo (Baris 108): Sistem tidak langsung memberikan diskon. Ada pengecekan ganda menggunakan operator && (DAN). Voucher hanya dianggap sah jika kodenya tepat "PROMO" dan tamu menginap minimal 3 malam.
    - Penerapan Diskon: Jika syarat di atas terpenuhi, sistem memberikan diskon sebesar 20% (dengan mengalikan total dengan 0.8). Namun, jika salah satu syarat gagal (misal: kode benar tapi cuma menginap 2 malam), sistem akan menolak penggunaan voucher dan mengembalikan harga normal

.

```
117     public void cetakInfo() {
118         System.out.println("===== STRUK INFORMASI KAMAR =====");
119         System.out.println("Nomor Kamar      : " + nomorKamar);
120         System.out.println("Tipe Kamar       : " + tipeKamar);
121         System.out.println("Kapasitas Maks   : " + kapasitasMaksimal);
122         System.out.println("Harga / Malam    : " + hargaPerMalam);
123         System.out.println("Status Tersedia  : " + isTersedia);
124         System.out.println("----------------------------------");
```

8. Pelaporan Data dengan cetakInfo() (Baris 117-124)

Setelah semua logika enkapsulasi dan validasi selesai, kita butuh cara untuk menampilkan hasil akhirnya.

- Tampilan Rapi: Metode cetakInfo() dirancang untuk merangkum seluruh kondisi atribut objek kamar saat ini.
- Akses Atribut: Di sini kita langsung memanggil variabel seperti nomorKamar, tipeKamar, hingga isTersedia. Karena metode ini berada di dalam kelas yang sama, ia bisa langsung mengakses atribut private tersebut untuk ditampilkan ke konsol.
- Fungsi Informatif: Metode ini sangat berguna saat proses debugging atau simulasi di kelas Main, karena kita bisa melihat apakah validasi yang kita buat (seperti perubahan otomatis tipe kamar atau harga) benar-benar sudah tersimpan di dalam objek atau belum.

Ini Adalah bagian Main.java:

```
1   public class Main {
2       public static void main(String[] args) {
3
4           KamarHotel kamar1 =
5                   new KamarHotel("101", "Reguler", 3);
6
7           kamar1.setTipeKamar("Presidential");
8           kamar1.setHargaPerMalam(-10000);
9
10          KamarHotel kamar2 =
11                  new KamarHotel("202", "Premium", 2, 300000);
12
13          kamar2.pesanKamar(4);
14
15          kamar2.pesanKamar(2);
16
17          kamar2.pesanKamar();
18
19          System.out.println("\nTagihan kamar1:");
20          double total1 = kamar1.hitungTotalBayar(2, "PROMO");
21          System.out.println("Total bayar: " + total1);
22
23          System.out.println("\nTagihan kamar2:");
24          double total2 = kamar2.hitungTotalBayar(4, "PROMO");
25          System.out.println("Total bayar: " + total2);
26
27          kamar1.cetakInfo();
28          kamar2.cetakInfo();
29      }
30  }
```

Pada baris 4 sampai 11, kita memulai simulasi dengan membuat dua objek kamar menggunakan dua jenis constructor yang berbeda. Kamar 1 didaftarkan dengan cara kilat, lalu kita langsung menguji ketahanan sistem dengan mencoba memasukkan tipe kamar "Presidential" dan harga negatif -10.000. Karena enkapsulasi yang kita buat sudah punya validasi ketat, sistem secara otomatis menolak input ngawur tersebut dan memaksanya kembali ke nilai standar, yaitu tipe "Reguler" dan harga 50.000. Sementara itu, Kamar 2 dibuat dengan pendaftaran lengkap yang memiliki kapasitas maksimal 2 orang.

Selanjutnya, pada baris 13 sampai 17, kita menguji logika pemesanan kamar (Method Overloading). Kita mencoba memesan Kamar 2 untuk 4 orang, dan sistem berhasil memblokirnya karena terjadi overcapacity. Setelah itu, kita coba memesan kembali dengan jumlah tamu yang pas (2 orang) dan sistem menyatakan berhasil. Untuk memastikan tidak ada double booking, kita coba panggil metode pesanKamar() lagi pada kamar yang sama, dan sistem memberikan peringatan kalau kamar tersebut memang sudah terisi.

Di bagian akhir (baris 19-28), kita mendemonstrasikan sistem penagihan yang cerdas. Pada Kamar 1, kita mencoba memakai voucher "PROMO" untuk menginap 2 malam, namun sistem menolak diskon karena tidak memenuhi syarat minimal 3 malam. Sebaliknya, pada Kamar 2, perhitungan untuk 4 malam dengan voucher yang sama berhasil mendapatkan potongan diskon 20%. Simulasi ditutup dengan memanggil cetakInfo() untuk menampilkan "Struk Informasi" akhir yang rapi, membuktikan bahwa semua atribut dari kedua objek kamar sudah berada dalam kondisi yang benar sesuai aturan bisnis yang kita tetapkan

IV. Kesimpulan

Secara keseluruhan, proyek sistem reservasi KamarHotel ini berhasil membuktikan bahwa penggunaan prinsip Enkapsulasi sangat efektif untuk menjaga integritas data dari kesalahan operasional maupun input yang tidak valid. Dengan menyembunyikan atribut penting dan menyaringnya melalui metode Setter, kita berhasil menciptakan sistem yang "kebal" terhadap human error. Selain itu, penerapan Method Overloading memberikan fleksibilitas tinggi pada program, sehingga satu sistem yang sama mampu menangani berbagai skenario nyata di lapangan, mulai dari pencegahan muatan berlebih (overcapacity) hingga validasi otomatis kode promo yang akurat.

IV. Daftar Pustaka

Tim Asisten Laboratorium Komputer (2026). Modul Praktikum Pemrograman Berorientasi Objek – Modul 4: The Core of OOP. Program Studi Ilmu Komputer, Universitas Pertamina

Tim Asisten Laboratorium Komputer (2026). Modul Praktikum Pemrograman Berorientasi Objek – Modul 5: Encapsulation. Program Studi Ilmu Komputer, Universitas Pertamina

Dosen PBO. (2026). The Core of OOP. Universitas Pertamina

Dosen PBO. (2026). Encapsulation. Universitas Pertamina